Session Storage Amplification in Long-Horizon Coding Agents: Measurement, Compaction, and a Reference-Based Persistence Architecture

An empirical study of 52.49 GiB of Codex session rollouts

Eng Rassam · October 2026 · Prepared for OpenAI

Abstract

Long-horizon coding agents persist a local record of each session so that work can be resumed: messages, tool activity, execution events, token counters and compacted replacements of the context. We measured every stored byte of 845 Codex rollout files (52.49 GiB, January–October 2026) from one developer's real work, without reading or reproducing any content.

- **What is stored.** The user-role and assistant conversation is 2.89% of stored bytes, or 0.64% without embedded images. Execution traces are about 15%. Compaction snapshots are 77.13%, and 69.34% of the corpus consists of images already present in the preceding snapshot.
- **Why it accumulates.** Each compaction re-stores the retained history in full: consecutive snapshots are about 98% identical. A compaction takes a median of 56.5 s, regardless of how much it writes.
- **Tokens.** Recorded token consumption follows the context sent and request continuity, not file size. After controlling for session length, file size has no association with tokens used.
- **Codex's October 2026 changes.** Pruning, parent references for subagents and zstd compression reduce the same files to 10.60 GiB. New snapshots still embed images.
- **A reference-based alternative.** Storing each image once in a media table shared across sessions, and referencing it, reduces the 843 comparable files to 3.00 GiB with the same zstd level: 3.53× below the current format.

We propose a layered, reference-based session store: a conversation log, a latest-checkpoint snapshot store, shared media, and an incrementally built project memory file. We give measured and expected effects, and recommendations for OpenAI.

Key Findings

1. **[O] Stored data versus conversation.** Stored files are 34.6× the conversation they record (median 14.7× per file). The cost is concentrated: the 100 largest files hold 91.9% of bytes (Section 7).
2. **[O] Repeated images.** Images repeated from one compaction snapshot to the next occupy 69.34% of all stored bytes. 106,260 embedded image occurrences encode only 3,969 distinct images, and stored base64 is 31.5× their decoded size (Sections 7.3, 8.3).
3. **[O] What a snapshot is.** It is 94.50% retained history, 90.21% images, and a 0.35% opaque summary. It is a rewrite of the history, not a condensation of it (Section 9.1).
4. **[O] Cost of compaction.** It is a full-context model call: 230,614 input tokens in the reconstructed turn, a median of 56.5 s, unrelated to snapshot size ($\rho = 0.089$). It is followed by a full resend (Sections 9.3, 10).
5. **[O] Tokens and file size.** Tokens are not driven by file size (partial $\rho = -0.019$, $n = 189$). Of logged usage rows, 27.1% are candidate duplicates, so summing logs overcounts (Section 10).
6. **[O] Subagent copies.** A subagent file shrank 124× once the parent history was referenced instead of copied. Seven of the ten largest files share its size signature (Section 11.4).
7. **[O] Codex's October 2026 changes.** They cut the corpus from 52.48 to 10.60 GiB, but leave image repetition in snapshots, which compression cannot reach (Section 12).

8. **[O] Measured alternative.** A shared, content-addressed media store yields 3.00 GiB, 3.53× smaller than Codex's current format. Referencing repeated history items adds almost nothing once compressed. The approaches are complementary (Section 14).
9. **[P] Priorities for OpenAI.** Three recommendations rank highest: shared content-addressed media, snapshots stored by reference with only the latest kept for resume, and parent references extended to all inherited history. Incremental, prefix-stable compaction and a project-level memory file come next (Section 17).

Contents

1. Introduction
1. Problem Definition
1. Related Work
1. Dataset, Ethics and Privacy
1. Measurement Taxonomy
1. Methodology
1. Storage Composition
1. Tool, I/O and Artifact Results
1. Compaction and Resume
1. Token Accounting and Its Relation to Stored Session Size
1. Duplication and Subagent Amplification
1. Before and After: Codex's October 2026 Storage Changes
1. Proposed Architecture: A Reference-Based, Layered Session Store
1. Quantitative Evaluation: References versus Compression
1. Security and Privacy Implications
1. Limitations
1. Recommendations for OpenAI
1. Conclusion
1. Appendices

1. Introduction

Coding agents such as Codex work in sessions that last hours or days. Throughout a session they persist what they need to resume it: user messages, model outputs, tool calls and their results, execution events, token counters and, whenever the context window fills, a compacted replacement for the history so far.

This record is written locally, append-only, as one JSONL file per session. On a machine used heavily for real software work it becomes large. The corpus studied here is 845 files and 52.49 GiB from nine months of one developer's work. Public reports describe stores of 91 GB and 755 GiB (Section 3).

Size is the visible symptom, but the underlying question is architectural. What does a long-horizon agent need to keep, in what form, and for which purpose: resuming, auditing, or informing the model? A record that mixes conversation, execution trace, working state and repeated snapshots of the context answers all three purposes at once, and serves each at the cost of the others.

This paper measures that record. Without reading or reproducing any conversation content, we took the following steps:

- indexed every physical record in the corpus;
- profiled each record type structurally;
- hashed every embedded image and text block in compacted snapshots;
- reconstructed token accounting where the logs allow it.

Codex changed its storage format during the study (rust-v0.164.0, October 2026). Because our corpus predates the change, we could compare the same sessions before and after it. Finally, we measured an alternative that stores content once and references it, against Codex's new format on the same files.

Contributions

1. **A byte-complete decomposition of agent session storage** (Section 7). Compacted snapshots hold 77.13% of stored bytes, and images repeated from one snapshot to the next alone 69.34%. The user-role and assistant conversation is 2.89%, or 0.64% without embedded images. Stored files are 34.6× the conversation they record (median 14.7× per file).
2. **An account of what compaction persists and costs** (Section 9):
   - a snapshot is 94.50% retained history and 90.21% images;
   - consecutive snapshots are about 98% identical;
   - a compaction takes a median of 56.5 s, unrelated to the size of what it writes, and is followed by a full resend.
3. **A duplication analysis** (Section 11). 97.85% of stored image bytes in compacted histories are repeats, mostly carried forward within a session. A subagent file shrank 124× once the parent history was referenced instead of copied.
4. **A before/after measurement of Codex's October 2026 changes** (Section 12). On 843 files, compression and a legacy rewrite reduce 52.48 GiB to 10.60 GiB. New snapshots, however, still embed images.
5. **A measured alternative** (Section 14). Images referenced from a media table shared across sessions, with the same zstd level, store the same 843 files in 3.00 GiB: 3.53× smaller than Codex's current format. The approach complements Codex's own pruning and parent references.
6. **A token-accounting analysis** (Section 10). Tokens follow the context sent and request continuity, not file size. After controlling for session length, file size has no association with tokens used (partial $\rho = -0.019$). 27.1% of logged usage rows are candidate duplicates.

7. **A proposed reference-based, layered session architecture** (Section 13), with recommendations for OpenAI (Section 17).

Scope

This is a measurement study of one user's corpus. It reports bytes, counts and recorded token counters; it does not estimate cost and does not generalize shares to other users. Each statement is labelled as observed, inferred, hypothesized or proposed (Section 5.5).

Structure

Sections	Content
5–6	The problem and related work
7–9	Data, definitions and method
10–14	Results
15–16	The proposal and its evaluation
17–20	Privacy, limitations, recommendations and conclusions

2. Problem Definition

2.1 One record, several purposes

A Codex session file serves at least four purposes:

Purpose	What it needs
Continuity	What the model must see to continue the task
Execution state	What the client must know to resume tools, turns and subagents
Audit	What a user or developer may want to inspect later
Telemetry	Counters and runtime events

Today all four live in one append-only stream of physical records. The question is not only how large that stream is, but which bytes serve which purpose, and which serve none.

2.2 Input, output, processing

As an analytical lens, and not as an assumption about Codex's implementation, we distinguish three kinds of content:

- **Input:** what the user or environment supplies, such as messages, pasted text, images and project instructions.
- **Output:** what the agent returns to the user as a response.
- **Processing:** what the agent generates while working, such as reasoning, tool calls and outputs, execution events, compaction records and state.

Section 5 turns this lens into byte categories that do not overlap. Two caveats from the data apply:

- the `user` role also carries input injected by the client;
- compaction records mix retained input with processing.

2.3 Context compaction is not storage compaction

Context compaction reduces what the model sees: it replaces history with a shorter retained context and a summary. **Storage compaction** would reduce what is kept on disk. The two are distinct, and in the corpus they work in opposite directions.

[O] In one reconstructed turn, compaction cut the next request's ordinary input from 230,382 to 34,868 tokens, while writing a new snapshot of 1,137,409 bytes to disk (Section 10). Each compaction adds a full snapshot, and the disk history grows with every context reduction.

2.4 Research questions

The research questions come from the project brief. The table shows where each is addressed and how far the evidence goes.

RQ	Question	Where	Status
RQ1	What occupies session storage, by bytes and by count?	§7, §8	Answered
RQ2	How large is storage relative to the conversation?	§7.4	Answered, with corpus-weighted and per-file ratios
RQ3	What does compaction persist and repeat?	§9	Answered for the stored form; the path used in practice is inferred
RQ4	What is required to resume?	§9.5	Structural answer only; resume memory and latency not measured
RQ5	Which tools and actions generate the most data?	§8	Answered by subtype; per-tool breakdown withheld for privacy
RQ6	How much is embedded content rather than referenced?	§8.3	Answered for images; pasted text not separated
RQ7	How much is duplicated?	§11	Answered for exact image, text and item duplicates
RQ8	Do subagents duplicate parent context?	§11.4	One case measured; corpus-wide pattern hypothesized
RQ9	Can conversation, working state, memory and trace be separated?	§13, §14	Proposed; storage effect measured, quality effect not
RQ10	What is exposed by local persistence?	§15	Described

2.5 What would count as an improvement

A better design should reduce stored bytes without losing what resume and audit need. It should also:

- keep the active context independent of disk history;
- avoid rewriting unchanged data;
- allow partial loading;
- let a user remove sensitive content in one place.

Section 14 evaluates storage directly. Resume cost, token use and context quality are stated as open measurements (Section 16).

3. Related Work

3.1 Context management for long-horizon agents

Context as a Tool. Liu, Yang et al. (*Context as a Tool: Context Management for Long-Horizon SWE-Agents*, arXiv:2512.22087, December 2025) criticize two common strategies: appending every interaction to the

context, and compressing only when the context is full. They argue that both lead to context explosion and semantic drift in long software-engineering tasks. Their alternative, CAT, has two parts:

- context maintenance becomes a tool the agent calls;
- context is held in a structured workspace with stable task semantics, condensed long-term memory and high-fidelity short-term interactions.

Relation to this paper. Their work concerns what the model sees. Ours concerns what the agent persists, and how persistence and context interact. Our measurements support their critique from the storage side: whole-context compaction in Codex rewrites about 98% unchanged history at every window (Section 9). Our proposal (Section 13) has a similar layered structure on disk:

- stable project memory;
- condensed checkpoints;
- recent items loaded in full.

3.2 Public reports on Codex session growth

Users have reported the same patterns in the public Codex issue tracker. These are individual reports, not controlled measurements, but they show that our corpus is not an isolated case.

- **openai/codex#24948**, "Codex session logs grow to 700MB-2GB from repeated compaction history and raw tool output", opened 2026-05-28.
  - The initial report describes a 732 MB rollout. Of it, 337.8 MB is in 684 `compacted` records and 230.8 MB in `function_call_output` records.
  - The reporter's sessions directory held about 91 GB.
  - Later comments report an 11.36 GiB rollout, about 10.68 GiB of it compacted records.

  This matches our findings on compaction dominance (Section 9) and tool outputs (Section 8).
- **openai/codex#34061**, "Insane Codex Disk Usage from Subagents", opened 2026-07-18.
  - About 755 GiB of session files, of which about 731.5 GiB are in 2,393 subagent files from one resumed parent.
  - The reporter hypothesizes that full-history forks copy the parent history into every child.

  Our measured case, a subagent file that became 124× smaller once Codex referenced the parent history, supports this mechanism (Section 11.4).

Both issues were open when we checked them (2026-10-28).

3.3 Codex's own response

The October 2026 release `rust-v0.164.0` introduced two changes:

- background zstd compression of idle rollouts;
- a rewrite of legacy rollouts that prunes superseded snapshots and references parent history from subagent files.

We treat this as the current state of practice and measure it directly (Section 12). Its reference mechanism for subagent history is the closest existing precedent for the reference-based design in Section 13.

3.4 Positioning

Prior work and public reports identify context growth and storage growth as problems. To our knowledge, this paper provides the first measurement that covers:

- every stored byte, across a large real corpus;
- the separation of conversation, processing, repetition and state;
- the cost of compaction in time and tokens;
- a before/after comparison of a vendor's own storage change;
- a reference-based alternative evaluated on the same files.

Scope of the claim. This statement is limited to the sources we reviewed. A systematic literature survey was out of scope.

4. Dataset, Ethics and Privacy

4.1 Corpus

[O] The primary dataset consists of 845 Codex rollout files (JSONL) copied from one researcher's local Codex store:

Measure	Value
Size	56,355,653,690 bytes (52.49 GiB)
Physical records	3,497,535
Period	2026-01-03 to 2026-10-04
Zero-byte files	none

(DATASET_INVENTORY.md, data/reports/session_statistics_2026.json)

Files per month of the session start:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct
1	18	18	93	50	33	43	226	326	37

[O] Clients and models. Clients, by the `originator` field of `session_meta`:

Client	Files
Codex Desktop	667
VS Code extension	168
terminal client	8
browser side panel	2

The first model observed in each file covers several model generations. The most frequent are the automatic review model (254 files) and five general models (44–163 files each). 112 files record no model.

[I] Representativeness. The corpus reflects one heavy user's work across clients and model generations. It is not a random sample of Codex users, and corpus-wide shares should be read as properties of this workload.

[O] File and session. A file is not necessarily one logical session: forks, resumes and subagents produce related files, and some file names carry compound identifiers.

4.2 Supporting material

- **Local state databases.** Copies of the six SQLite stores that Codex keeps beside the rollouts hold 28 tables and 11,728 rows. One backup copy is corrupt and was excluded (*SQLITE_SCHEMA.md*). Metadata from 9 tables, 11,714 rows, was imported into the research database with text values replaced by lengths and hashes (*sqlite_metadata_import_v1/phase2_summary.json*).

- **Before/after pairs.** For the three largest corpus files, copies of the compressed `.zst` files that Codex produced in October 2026 (Section 12).
- **Recent sessions.** Copies of four sessions active between 2026-10-16 and 2026-10-28, used only for structural checks (Section 9.4).
- **Public source code.** The `openai/codex` repository at tag `rust-v0.164.0` (commit `6aaac69`).

4.3 Ethics and data handling

The data are the researcher's own working sessions, analysed by the researcher for this study. The following rules applied throughout:

1. **Copies only.** Analysis ran on copies. The live Codex store was accessed only to list file names and sizes and to copy specific files, each with explicit approval.
2. **Read-only sources.** Source files were never modified, moved, compressed or deleted. Size and modification time were verified before and after each scan.
3. **No content in outputs.** Reports, databases and this paper contain counts, byte sizes, hashes, locators, field paths and short enumerations only. No conversation text, image, command, path, title or account identifier is reproduced.
4. **Images are never viewed.** Images were hashed, and where needed decoded to measure size. They were never rendered, saved as images or compared visually.
5. **Local processing.** No project data was sent to an external service. The only external material consulted was public source code and public documentation.
6. **Step-by-step approval.** Each scan, database write or access to a new scope was proposed, approved and then run, with its scope recorded in the project log.

4.4 Release

The published material consists of this paper and the aggregate reports. The session files, samples, database copies and any file listing session identifiers or names are not released.

5. Measurement Taxonomy

5.1 Units of observation

Term	Meaning
File	One rollout JSONL file. Not necessarily one logical session.
Physical record	One line of a file. Records replayed on resume or copied from a parent are counted each time they are stored, so record counts are not message, request or action counts.
Record type	The top-level `type` field: `compacted`, `response_item`, `event_msg`, `world_state`, `session_meta`, `turn_context`, `token_usage_record`, `inter_agent_communication_metadata`.
Subtype	`payload.type` within a record type, e.g. `reasoning` or `function_call_output` in `response_item`, or `token_count` in `event_msg`.
Snapshot	One `compacted` record, with its `replacement_history` (the retained context) and optional `guardian_history` (history kept for the safety reviewer).
Item	One element of a snapshot's history arrays.

5.2 Functional categories

Every stored byte is assigned to exactly one category, so the categories sum to the corpus size (Section 7.3; *storage_composition_v1*):

Category	Members
Conversation	`response_item` messages with role `user` or `assistant`. The `user` role includes context injected by the client and does not establish human authorship.
Processing	`response_item` reasoning, tool calls and tool outputs; `event_msg` execution and runtime events.
Logging copies	`event_msg` records that repeat conversation messages (`user_message`, `agent_message`, and message items of `item_completed`).
Token counters	`event_msg/token_count`.
State and metadata	`world_state`, `session_meta`, `turn_context`, `token_usage_record`, inter-agent metadata.
Compacted	All `compacted` bytes, further split into: images present in the preceding snapshot, text blocks present in the preceding snapshot, and all other bytes.
Other	Remaining `response_item` subtypes (developer messages, `compaction`, `agent_message`, `ghost_snapshot`).

5.3 Duplication

- **Exact identity only.**
  - Images are identical when the SHA-256 of their decoded bytes matches.
  - A text block is one selected JSON string, compared by SHA-256 of its unescaped UTF-8.
  - An item is compared by its exact serialized bytes.

  Re-encoded, resized or reworded content counts as distinct.
- **Retained copy.** For duplicate accounting, the first occurrence in file, line and JSON traversal order is retained; all later occurrences are duplicates.
- **Carry-forward.** An occurrence also present in the immediately preceding snapshot of the same file. This is a subset of within-file duplication.
- **Stored versus decoded bytes.** Stored bytes include JSON quoting, escaping and data-URL prefixes; decoded bytes are the binary image size. The two are never subtracted from each other.

5.4 Tokens

- **Counters.** Token counters are read only where Codex recorded them: input, cached input, cache-write input, output, reasoning output and total.
  - total = input + output; cached $\subset$ input; reasoning $\subset$ output.
  - A missing counter is unknown, not zero.
- **Scopes.** Response, turn, thread, last and cumulative scopes are alternative views of the same consumption and are never added together.
- **Duplicate flags.** A row flagged as a possible duplicate is a candidate, not evidence of a duplicate charge.
- **Bytes are not tokens.** No storage share is converted into a token share, and no cost is computed.

5.5 Statement labels

Label	Meaning
[O] Observed	Measured directly from the data or read from public code.
[I] Inferred	Follows from observations by stated reasoning.
[H] Hypothesized	Consistent with observations but not verified.
[P] Proposed	A design or recommendation.

Each statement also identifies its period: the pre-October corpus, the October 2026 code, or the post-October samples.

5.6 Units

Unit	Bytes
GB	10^9
GiB	2^{30}
MB	10^6
MiB	2^{20}

Corpus-weighted ratios divide summed numerators by summed denominators. Per-file statistics (median, P10, P90) describe the typical file. Unweighted per-file means are not used for conclusions.

6. Methodology

6.1 Pipeline

The analysis ran as a sequence of separately approved stages. Each stage writes a versioned report folder (`data/reports/<analysis>_v<N>/`) and a dated entry in the project log. A new version never overwrites an earlier one.

Stage	Method	Output
1. Inventory	File-system metadata only: names, sizes, timestamps	`DATASET_INVENTORY.md`
2. Record index	One sequential pass in 1 MiB chunks. Stores one locator per physical line (file, line number, byte offset, byte length, timestamp text, top-level type), keeping at most 64 KiB of a line's start	PostgreSQL `research_session_records` (3,497,535 rows)
3. Type repair	535 untyped records traced to a UTF-8 decode cut inside a multibyte character; repaired from 35 MB of record headers	`null_record_type_repair_v1.json`, `STRUCTURAL_ANALYSIS_STEPS_1_3.md`
4. Structural censuses	Streaming parsers over indexed byte ranges of `response_item`, `event_msg` and `compacted`. They export only field paths, JSON types, short allow-listed enumerations, counts and byte sizes	`*_profile_v1`, `compacted_item_sizes_v1`
5. Hash indexes	Images decoded incrementally and hashed (SHA-256); text blocks hashed after JSON unescaping. Indexes hold hashes, lengths and locators in SQLite	`compacted_image_duplicates_v1`, `compacted_text_duplicates_v1`
6. Growth and composition	Queries over locators and hash indexes: snapshot growth, carry-forward, functional decomposition	`compaction_growth_v1`, `storage_composition_v1`
7. Token observations	Every JSON object carrying a token counter, in every record type, with exact locator and identifiers. Duplicate candidates flagged by rule, never deleted	`research_token_usage_observations` (1,778,473 rows); `token_usage_duplicate_candidates_v1`
8. Token case studies	Turn timeline, output components, request bodies from the local trace log, storage–token association with length control	`turn_token_timeline_v1`, `output_token_components_v1`, `logs_request_capture_v1`, `tokens_vs_storage_v1`
9. Local state	SQLite stores exported and imported, with text values reduced to lengths and hashes	`SQLITE_SCHEMA.md`, `sqlite_metadata_import_v1`
10. Before/after	Codex `.zst` copies decompressed and compared record by record with pre-change copies; Codex `.zst` sizes listed by name	`zstd_comparison/`, `zst_counterpart_sizes_v1`
11. Reference variants	Each file streamed once; variants compressed in memory; byte counts only	`media_reference_comparison_v1`, `media_reference_comparison_v2`

6.2 Resource discipline

Every script follows the same rules:

- **Streaming.** Files are read in 256 KiB chunks (1 MiB for the record index). No whole file, history or long string is loaded into memory. Where an item had to be held in memory, its maximum size is reported (238

KB in Section 14).

- **Scheduling.** One process at below-normal priority, with no parallel workers.
- **Restart.** Atomic per-file checkpoints allow a run to resume.
- **Integrity.** Source size and modification time are verified before and after each file.
- **Failure handling.** Parse errors, unknown structures and limit hits stop the run with an error. Nothing is skipped silently and reported as complete.
- **Measurement.** Runtime and peak working set are recorded in each report. For example, the corpus-wide reference evaluation read 52.48 GiB in 274 s with a peak working set of 42 MiB.

6.3 Verification

- **Synthetic tests.** Every analyzer has unit tests on synthetic fixtures. They cover chunk boundaries, escapes, truncated lines and byte-exact reconstruction for the reference variants. No test reads research data.
- **Reconciliation.**
  - Per-type bytes sum to the corpus size.
  - Functional categories sum to the corpus size.
  - Compacted partitions sum to the compacted total.
  - Independent analyzers agree, for example on 8,363 snapshots, 1,424,366 history items and 99,244 images in history items.
- **Calibration.** Python's zstd level 3 reproduced the size of Codex's own `.zst` files byte for byte, so the compressed comparisons use the same compressor settings.

6.4 Software

- Python standard library only; no third-party packages. Python 3.14 for `compression.zstd`.
- PostgreSQL 18 for locators and observations: migrations `001`–`006`.
- SQLite for hash indexes and copied Codex stores.

Commands, limits and caveats for every script are listed in `scripts/README.md` (Appendix).

6.5 Limits of the method

- One user's corpus.
- Exact-match duplication only.
- Client timestamps for durations.
- No server-side token data.
- No direct observation of which compaction path ran.
- Resume memory and latency were not measured.

7. Storage Composition

This section answers RQ1 (what occupies session storage) and RQ2 (how much larger the stored files are than the conversation they record).

All figures refer to the 845 rollout files in the study corpus, which cover January to October 2026 and predate Codex's October 2026 storage rewrite. Report paths are relative to `data/reports/`. Statement labels: **[O]** observed, **[I]** inferred, **[H]** hypothesized, **[P]** proposed.

7.1 Corpus size and concentration

[O] The corpus holds 845 files totalling 56,355,653,690 bytes (52.49 GiB) and 3,497,535 physical records (*session_statistics_2026.json*).

Sizes are highly skewed:

Measure	Value
Median file	2.15 MB
Mean file	66.7 MB
90th percentile	118.1 MB
99th percentile	984.0 MB
Largest file	8.53 GB

Concentration	Share of corpus bytes
Largest file	15.13%
10 largest files	31.40%
100 largest files	91.87%
89 files above 100 MB	90.31%
4 files above 1 GB	20.88%

(dataset_inventory.csv; computed with the standard library from the inventory, no file contents read)

[I] Storage cost is therefore a property of a minority of long sessions rather than of typical sessions. Corpus totals are dominated by these files, and per-file medians describe the typical session. We report both throughout.

7.2 Bytes by record type

[O] Every physical record carries one of eight top-level types (*record_type_counts_corrected_v1.json*):

Record type	Records	Bytes	Share of bytes
`compacted`	8,363	43,464,644,976	77.13%
`response_item`	1,490,433	7,030,455,815	12.48%
`event_msg`	1,849,181	4,758,495,586	8.44%
`world_state`	11,853	449,686,946	0.80%
`session_meta`	10,562	324,813,399	0.58%
`turn_context`	52,095	261,993,740	0.46%
`token_usage_record`	72,324	65,230,877	0.12%
`inter_agent_communication_metadata`	2,724	332,351	<0.01%
Total	**3,497,535**	**56,355,653,690**	**100%**

- The per-type bytes sum exactly to the corpus size.
- No record has a NULL type: 535 records that an early indexer left untyped, because of a UTF-8 chunk-boundary bug, were repaired (*null_record_type_repair_v1.json*).

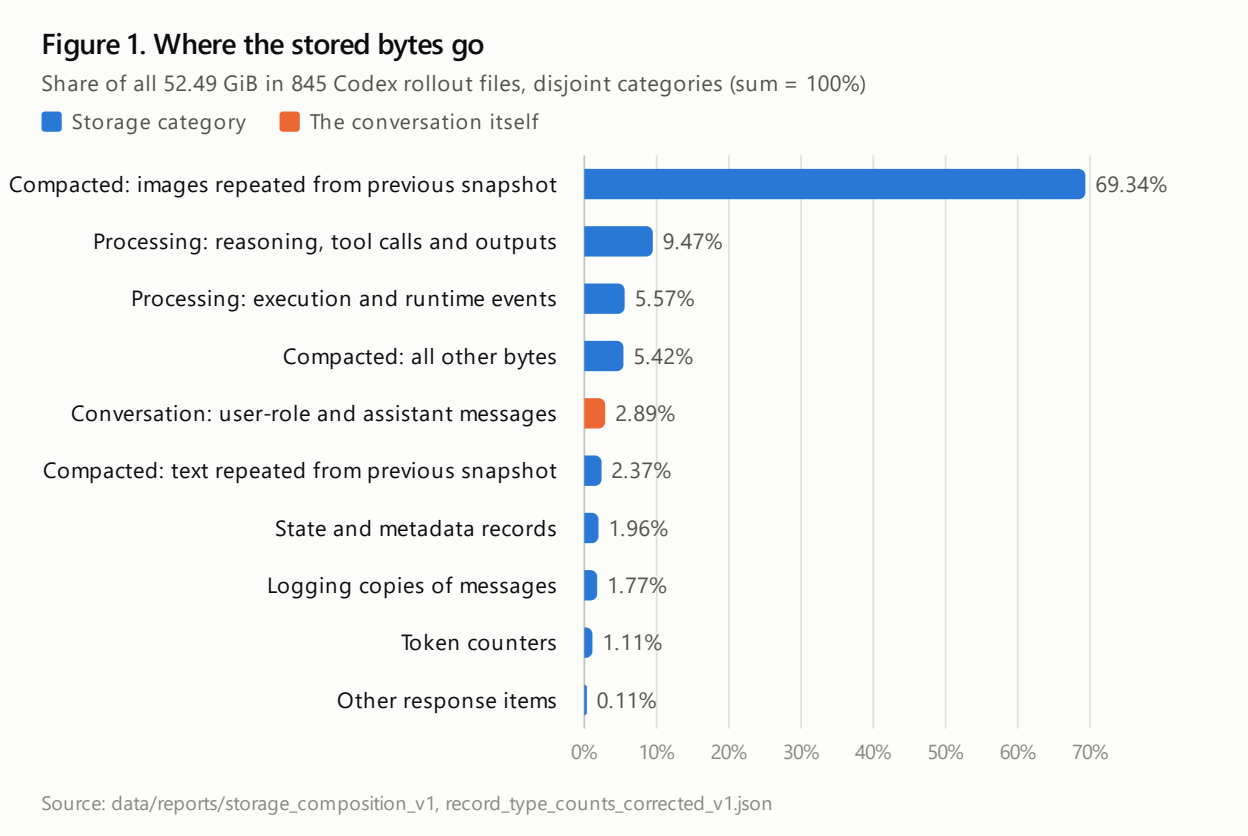
[O] Compacted records make up 0.24% of records but 77.13% of bytes. They occur in 434 of the 845 files (*compacted_item_sizes_v1*).

7.3 Functional decomposition

Record types mix functions. For example, `event_msg` carries both copies of conversation messages and execution events. We therefore partitioned all bytes into functional categories that do not overlap

(*storage_composition_v1*; categories as defined in its classification table).

Category	Bytes	Share
Compacted: image identical to one in the preceding compacted record	39,077,030,034	69.34%
Compacted: text block identical to one in the preceding compacted record	1,335,546,618	2.37%
Compacted: all other bytes	3,052,068,324	5.42%
Processing: reasoning, tool calls and tool outputs (`response_item`)	5,339,656,937	9.47%
Processing: execution and runtime events (`event_msg`)	3,136,268,147	5.57%
Conversation: user-role and assistant messages (`response_item`)	**1,627,855,999**	**2.89%**
Logging copies of messages (`event_msg`)	995,650,129	1.77%
Token counters (`event_msg/token_count`)	626,577,310	1.11%
State and metadata (`world_state`, `session_meta`, `turn_context`, `token_usage_record`, inter-agent metadata)	1,102,057,313	1.96%
Other `response_item` (developer messages, `compaction`, `agent_message`, `ghost_snapshot`)	62,942,879	0.11%
Total	**56,355,653,690**	**100%**



- **[O] One category dominates.** Images that already appeared in the preceding compacted record occupy 69.34% of the corpus. Conversation messages occupy 2.89%.
- **[O] The conversation baseline is mostly images.** The 1,627,855,999 bytes of user-role and assistant messages contain 1,269,084,976 bytes of embedded base64 image characters. Without them, the message text is 358,771,023 bytes. *(storage_composition_v1)*
- **Caveats.**
  - `user` role does not establish human authorship. It includes context injected by the client, such as project instructions and environment descriptions.
  - "Identical to the preceding record" counts only repetition between consecutive compacted records. Older repeats remain in the "all other bytes" row.

7.4 Amplification relative to the conversation

We express stored bytes as multiples of the conversation baseline, i.e. user-role and assistant messages (*storage_composition_v1*):

Ratio	Corpus-weighted	Per-file median	Per-file P90
A1: all bytes ÷ conversation	34.62×	14.73×	99.74×
All bytes ÷ conversation text without image base64	157.08×	20.77×	414.94×
A2: processing bytes ÷ conversation	5.21×	2.62×	47.86×
A3: compacted bytes ÷ conversation	26.70×	0.67×	57.69×

- Corpus-weighted ratios divide summed numerators by the summed baseline.
- 53 files whose baseline is zero are excluded from per-file distributions.
- Unweighted per-file means are dominated by files with a small baseline and are not used here.

[O] Amplification is concentrated. In the typical file, compacted bytes are smaller than the conversation (median A3 = 0.67×). Corpus-wide they are 26.70×, because a minority of long sessions accumulate many snapshots.

[I] Long sessions grow from compaction. Amplification in long sessions comes mainly from compaction rather than from execution. The 9.47% + 5.57% of processing bytes are what a coding agent is expected to log; the 69.34% of repeated snapshot images are not required to resume, since each snapshot replaces the history before it (Section 9).

7.5 Summary

- **[O]** Compacted snapshots are 77.13% of stored bytes, and images repeated from the preceding snapshot alone are 69.34%. The conversation is 2.89% of stored bytes, and only 0.64% without embedded images.
- **[O]** Stored files are 34.62× larger than the conversation they record (median 14.73× per file).
- **[I]** The cost lies in how snapshots and images are persisted, not in conversation volume, and it is concentrated in long, image-bearing sessions.

Limits. These are byte measurements of physical records and do not translate into token counts or cost. Record counts include histories replayed on resume and do not count messages or actions. The corpus comes from one user's machine and is not a random sample of Codex users.

8. Tool, I/O and Artifact Results

This section covers RQ5 (tool and execution patterns) and RQ6 (embedded content). Scope, paths and labels follow Section 7.

8.1 Execution trace in `response_item`

[O] Reasoning and tool activity make up 5,339,656,937 bytes (9.47% of the corpus) (*response_item_profile_v1*):

Payload type	Records	Bytes	Mean bytes per record
`custom_tool_call_output`	263,541	2,598,133,755	9,859
`function_call_output`	200,055	1,135,397,507	5,675
`reasoning`	383,510	983,123,414	2,563
`custom_tool_call`	263,545	511,775,048	1,942
`function_call`	200,102	109,019,189	545
`image_generation_call`	1	1,368,878	—

Payload type	Records	Bytes	Mean bytes per record
`tool_search_output`	78	684,104	8,771
`web_search_call`	417	133,012	319
`tool_search_call`	78	22,030	282

[O] Tool outputs.

- Outputs total 3,733,531,262 bytes, about 6.0× the bytes of the calls that produced them (620,794,237).
- 926 `custom_tool_call_output` records contain a string longer than 65,536 characters.
- Tool outputs also carry images: 2,532 image parts in 1,327 `custom_tool_call_output` records and 229 image parts in `function_call_output`.

[O] Reasoning.

- Every one of the 383,510 reasoning records carries `encrypted_content`.
- 130,884 of them also carry readable summary text.
- Reasoning bytes are therefore mostly opaque to the client. We did not measure the split between encrypted and summary bytes.

8.2 Event stream (`event_msg`)

[O] The event stream holds 1,849,181 records and 4,758,495,586 bytes (*event_msg_profile_v1*). The largest subtypes:

Subtype	Records	Bytes
`item_completed`	185,241	2,207,768,503
`user_message`	37,543	759,180,048
`token_count`	779,335	626,577,310
`patch_apply_end`	108,198	549,946,773
`exec_command_end`	52,050	228,975,358
`agent_message`	149,697	120,395,058
`task_complete`	40,816	80,431,521
`thread_settings_applied`	34,918	75,791,423
`agent_reasoning`	400,566	62,030,480
`mcp_tool_call_end`	1,557	14,527,092

Of the event-stream bytes (*storage_composition_v1*):

Part	Bytes	Description
Logging copies	995,650,129	`user_message`, `agent_message`, and `item_completed` items of type user or agent message; these repeat content also stored as `response_item`
Execution and runtime events	3,136,268,147	
Token counters	626,577,310	779,335 snapshots

[I] Execution is logged twice. Execution appears in both streams: as model-facing items in `response_item` and as client events in `event_msg`. For example, `patch_apply_end` and `exec_command_end` record the outcome of calls already present as tool outputs. We did not measure how much of this overlaps byte for byte.

8.3 Embedded images

[O] Corpus-wide count. A read-only pass replaced every embedded data-URL image with a reference and hashed it (*media_reference_comparison_v2*):

Measure	Value
Image occurrences	106,260
Base64 characters	43,635,193,249 (77.4% of corpus bytes)
Distinct images	3,969
Distinct images, decoded binary	1,384,078,592 bytes
File-image pairs (one image per file)	6,368, holding 2,012,406,757 decoded bytes

[O] Ratio. Stored base64 is 31.5× the decoded size of the distinct images it encodes.

[O] Where the repetition is. Within compacted records alone (*compacted_image_duplicates_v1*):

Measure	Value
Image appearances	99,244
Distinct images	1,936
Decoded size of distinct images	0.61 GiB
Repetition within the same file (an image carried into later snapshots)	35.43 GiB
Repetition across files	0.70 GiB

[I] What this means.

- Image storage is dominated by re-serialization within a session, not by the number or size of images users provide.
- A content-addressed media store captures most of the reduction within a file, and a store shared across sessions adds a further 628 MB, or 31% of per-file media bytes (Section 14).

8.4 Summary

- **[O]** Tool outputs (3.73 GB) are the largest part of the execution trace, ahead of reasoning (0.98 GB, mostly encrypted).
- **[O]** The event stream repeats about 1.0 GB of messages already stored as `response_item`.
- **[O]** Embedded images amount to 43.64 GB of base64 for 1.38 GB of distinct image data.

Limits.

- Tool names and arguments were not exported, by design (privacy), so we do not break outputs down by tool.
- Overlap between `event_msg` execution events and `response_item` tool outputs was not measured byte for byte.
- Image identity is exact hashing of the base64 payload.

9. Compaction and Resume

This section covers RQ3 (compaction behaviour) and RQ4 (resume requirements). Corpus figures describe January–October 2026 files, before the October 2026 rewrite. Code references are to the public `openai/codex` repository at tag `rust-v0.164.0` (commit `6aaac69`). Labels and paths follow Section 7. Token accounting is treated separately in Section 10.

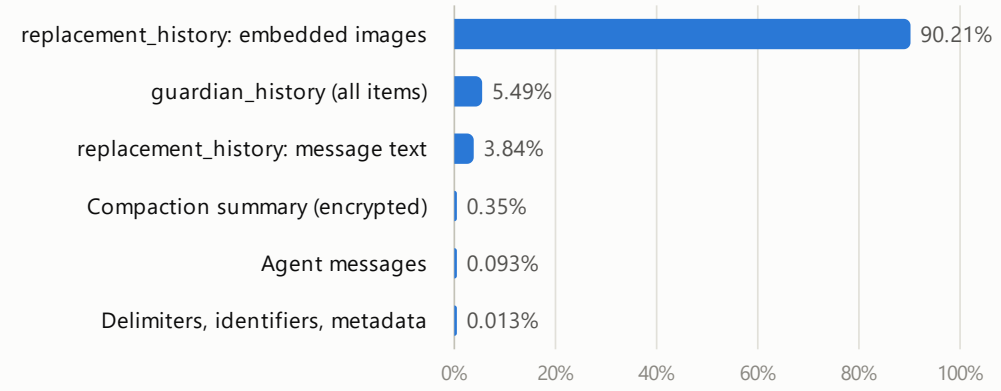
9.1 What a compacted record contains

[O] A compacted record is the client's persisted replacement for the conversation so far (*compacted_item_sizes_v1*, *compacted_profile_v1*). Across 8,363 records (43,464,644,976 bytes):

Part	Bytes	Share of compacted bytes
`replacement_history`: the context that replaces history	41,073,238,627	94.50%
— of which embedded images in messages	39,210,063,415	90.21%
— message text and structure	1,667,585,210	3.84%
— `compaction` item (encrypted summary, one per record)	153,735,880	0.35%
— `agent_message` items	40,621,567	0.09%
`guardian_history`: history kept for the safety reviewer (1,029 records)	2,387,178,293	5.49%
Identifiers, window numbers, usage snapshot and resume metadata	4,228,056	0.01%

Figure 2. What a compacted snapshot contains

Share of all compacted bytes (8,363 snapshots, 40.48 GiB)



Source: data/reports/compacted_item_sizes_v1

[O] Items in `replacement_history`. It holds 1,224,192 items, 146 per record on average:

Item	Count
User-role messages	1,149,276
Developer messages	16,916
Assistant messages	19,033
Agent messages	30,604
`compaction` summary items, one per record	8,363

Message content parts include 93,599 embedded images. Reasoning, tool calls and tool outputs are not retained there; they appear only in `guardian_history`.

[O] Summaries are opaque. The summary is stored as encrypted content and cannot be read on the client.

[I] What a compacted record is. It is not a condensed conversation. It re-serializes the retained user-role messages, including their images, and appends a small, opaque summary.

9.2 Growth and repetition across snapshots

[O] Snapshot counts. Of the 845 files, 434 contain compacted records (*compaction_growth_v1*):

Compacted records per file	Files
1	158

Compacted records per file	Files
2–5	97
6–10	34
11–25	63
26–50	30
51–100	18
101 or more	34

[O] Growth.

- Across 7,929 consecutive pairs, 4,643 snapshots grew and 3,286 shrank.
- Per file, the sum of snapshot sizes divided by the last snapshot's size has a median of 2.864, a P90 of 42.8 and a maximum of 664.7.
- In the largest files, snapshots plateau at about 9.15 MB each, with 130 to 199 snapshots per file.

[O] Repetition between consecutive snapshots. We split compacted bytes into three parts that do not overlap (*compaction_growth_v1/text_growth.md*):

Part	Bytes	Share of compacted
Images that also appeared in the preceding compacted record	39,077,030,034	89.91%
Text blocks that also appeared in the preceding compacted record	1,335,546,618	3.07%
Everything else (first appearances, other bytes)	3,052,068,324	7.02%

Of 99,244 image appearances, 86,841 are first copies of an image carried over from the preceding snapshot; only 5 skip a snapshot.

[O] Sample. In three consecutive mid-session snapshots of 9.1 MB each:

- 270 to 271 of 275–279 items were byte-identical to the previous snapshot;
- 6 to 8 new items were appended at the tail;
- 5 items were dropped, the previous summary among them.

User-role messages made up 9.19–9.20 MB of each record and carried 24 images (*zstd_comparison/compaction_snapshot_sample.md*).

[I] Rewriting, not condensing. Each compaction rewrites the retained history in full, about 98% of it unchanged, rather than recording what changed.

9.3 Duration and token cost of compaction

[O] Duration. 755 compactions carry both start and end timestamps in `item_completed` events of type `ContextCompaction` (*compaction_cost_v1*):

Measure	Value
Median duration	56.5 s
P90 duration	123.2 s
Spearman correlation with the size of the resulting snapshot (n = 697)	0.089

[O] Token readings. The latest-usage snapshots attached to 74 compacted records show a median of 244,402 input tokens and 783 output tokens. In one fully reconstructed turn, the compaction response itself used 230,614 input tokens (99.8% cached) and 2,826 output tokens. The first request afterwards sent 16,820 uncached input tokens (Section 10; *turn_token_timeline_v1*).

[I] Cost. Compaction is a full-context model call taking about a minute. Its duration is not explained by the size of what is written to disk, so the cost lies in the call, not in the write.

9.4 When compaction runs, and which path

[O] Triggers (`codex-rs/core/src/session/turn.rs`). Compaction is evaluated against the model selected for the current turn, using its `auto_compact_token_limit` and context window:

Trigger	When it runs	Reason recorded
Pre-turn	the limit is already reached before sampling	—
Mid-turn	the limit is reached while the turn continues	—
Post-turn	an optional percentage threshold is reached	—
Model switch	the new model has a smaller window than the previous one and the active context exceeds the new limit; compaction then runs with the previous model	`ModelDownshift`

If the request still exceeds the window (`ContextWindowExceeded`), the turn ends with an error rather than compacting, except in the guardian case.

[O] Two compaction paths. The path depends on the provider's capabilities:

- **Local** (`compact.rs`).
  - Rebuilds the history from three parts: the re-rendered initial context, the most recent user messages within `COMPACT_USER_MESSAGE_MAX_TOKENS = 20_000`, and the summary.
  - Drops images and audio.
- **Remote v2** (`compact_remote_v2_installation.rs`).
  - Installs a compaction output produced on the server.
  - Applies a retained-image budget only when `Feature::CompactionImageBudget` is enabled.

[O] Recent sessions. Four sessions active between 2026-10-16 and 2026-10-28 still write a full `replacement_history` with embedded images at every compaction, including after the 2026-10-23 release. In the last snapshot of one session, 246 of 258 items were identical to the previous snapshot (*post_october_compaction_sample_v1*).

[H] Path in practice. These sessions most likely used the remote path, which retains images. The client version that wrote each snapshot is not recorded, so this is not verified.

9.5 What resume requires

- **[I] Only the latest snapshot.** Each snapshot replaces the history before it, so reconstructing context needs only the latest snapshot plus the records after it. Earlier snapshots are needed only for audit.
  - Codex's October rewrite is consistent with this: it removed 163 of 190 snapshots from the largest file while keeping all 220 distinct images (*zstd_comparison/stage2_summary.json*).
- **[O] Appending to an old session.** Since October 2026, appending to a session idle for more than 7 days first decompresses the whole `.zst` file back to plain JSONL (`materialize_rollout_for_append`, `codex-rs/rollout/src/compression.rs`).
- **Not measured:**
  - resume memory and latency, before or after the October changes;
  - whether a rewritten rollout reconstructs exactly the same model context as the original.

9.6 Summary

- **[O] Contents.** A compacted record is 94.50% retained history, which is 90.21% images; the summary is 0.35%, opaque.
- **[O] Repetition.** 89.91% of compacted bytes are images already present in the preceding snapshot, and consecutive snapshots in the sample were about 98% identical.
- **[O] Cost.** Compaction takes about a minute (median 56.5 s), unrelated to snapshot size, and is followed by a full resend.
- **[I] Implication.** The storage cost of compaction comes from how its result is persisted, and its time and token cost come from summarizing the full context. Both point to persisting changes and references instead of full snapshots, and to incremental rather than whole-context summarization (Sections 13 and 17).

Limits.

- Durations rely on client timestamps, available for 755 of 1,314 compaction events.
- Token readings cover a small set of diagnostic cases and are not corpus statistics.
- Item and text identity is exact bytes.
- The code analysis covers one public release, and the path used in practice was not observed directly.

10. Token Accounting and Its Relation to Stored Session Size

This section answers how recorded token consumption relates to stored content. Every figure cites the report it comes from (paths relative to `data/reports/`). Statement labels: **[O]** observed, **[I]** inferred, **[H]** hypothesized, **[P]** proposed.

Summary

Local session storage and model-token consumption are governed by different mechanisms. Codex persists the full interaction history on disk, including repeated copies of embedded images, but in the ordinary case it transmits only the new input items of each request and relies on server-side continuation of the previous response. Recorded input tokens therefore track the size of the context the server processes, most of which is reported as cached, rather than the number of bytes written to or read from the session file. Across 189 linked session files, file size and token totals are strongly rank-correlated, but the association disappears once session length is controlled. Within this evidence, storage amplification is a disk, memory and parsing cost rather than a direct driver of token consumption.

10.1 What the counters measure

- **[O]** Each usage record exposes six counters: `input_tokens`, `cached_input_tokens`, `cache_write_input_tokens`, `output_tokens`, `reasoning_output_tokens` and `total_tokens`. Where all are present, total = input + output, cached input is a subset of input and reasoning output is a subset of output. *(turn_token_timeline_v1)*
- **[O]** Input dominates. In six diagnostic files, input accounted for 99.2–99.8% of total tokens within verified counter intervals. *(storage_token_case_study_v1)*
- **[O]** In one fully reconstructed turn (26 model responses, 271.9 s), the responses consumed 2,478,954 input tokens, of which 2,407,808 (97.13%) were reported as cached, and 12,955 output tokens, of which 3,977 were reasoning. These are sums across invocations, not the size of any single context window. *(turn_token_timeline_v1)*

- **[O]** The `threads.tokens_used` field in Codex's state database equals the last cumulative `total_tokens` reading of the corresponding session file in 179 of 180 comparable files (99.44%). The one exception equals an earlier reading in the same file. `tokens_used` therefore includes cached input. *(tokens_vs_storage_v1/report.md)*
- **[O]** Event-stream cumulative counters can differ from response-level sums. In the reconstructed turn, the event cumulative delta was lower by exactly the compaction response's own usage (233,440 tokens). Counter scopes must not be mixed or added. *(turn_token_timeline_v1)*

10.2 What is sent versus what is stored

- **[O]** Session files do not store the request sent to the model. A structural audit found no request bundle in a fully parsed file, and the corpus-wide structural censuses contain no `previous_response_id`, `tool_choice` or `messages` keys. *(model_request_capture_audit_v1; response_item_profile_v1; event_msg_profile_v1; compacted_profile_v1)*
- **[O]** Trace logs from a separate Codex log database (16 conversations, 14–18 June) contain 27 structured requests. Twenty reference `previous_response_id` and carry only new input items. Nineteen of these could be paired with a response by order within the conversation. *(logs_request_capture_v1/report.md, phase4.md)*
- **[O]** The incremental requests are small but report large inputs. Their logged JSON ranged from 22,253 to 87,150 bytes, with a median of 23,790. The paired responses reported 20,903–133,744 input tokens, with a median of 109,292, and a median cached share of 98.3%. For 17 of the 19, the cached share was 90.8–99.7%. In one example, a request carrying 97 bytes of input items reported 97,548 input tokens, of which 97,152 were cached. *(logs_request_capture_v1/phase4.md)*
- **[O]** The public Codex client (rust-v0.164.0, commit `6aaac69`, `codex-rs/core/src/client.rs`) implements this behavior. `get_incremental_items` sends only the items that extend the previous request plus its output, together with `previous_response_id`. The client falls back to full input in these cases:
  - no previous response is available;
  - request properties (model or tools) changed;
  - the input was shortened or does not match;
  - the WebSocket connection was closed;
  - transport fell back to HTTP, for example on status 426.

  The October release is not established as the build that produced the June logs. *(logs_request_capture_v1/phase4.md; independently checked)*
- **[I]** Under incremental continuation, input tokens measure context reconstructed by the server, not bytes transmitted by the client. Local byte counts cannot stand in for token counts.
- **[O]** Full-input requests also occur. In one conversation, five consecutive requests each resent the whole history, growing from 862 to 879 items at about 1.95 MB of logged JSON each. Every one of them contained the same image of 528,812 decoded bytes. No token readings were captured for these five. *(logs_request_capture_v1/phase4.md)*
- **[O]** One full-input request of 4,350,704 logged bytes, containing three images, reported 58,781 input tokens (31,616 cached). Request bytes and input tokens can differ by orders of magnitude in either direction. *(logs_request_capture_v1/phase4.md)*

10.3 Images: stored copies versus transmitted copies

- **[O]** On disk, embedded image strings occupy 38.03 GiB of compacted records, but represent only 1,936 unique images (0.61 GiB of binary data). 35.43 GiB of the repeated copies come from the same image being re-stored in later compacted records of the same file. *(compacted_image_duplicates_v1)*

- **[O]** Of the four unique images found in logged requests, three are byte-identical to images stored in compacted histories of the same session files (7 of 9 request occurrences). *(logs_request_capture_v1/report.md)*
- **[I]** The images retained in history are transmitted again when the full input is resent. Under incremental continuation they are not part of the transmitted delta. The token cost of an image cannot be derived from its stored byte size and is not reported per item.

10.4 Storage size versus token totals across files

Cohort: 189 session files linked to the state database whose last record precedes 19 June. *(tokens_vs_storage_v1/report.md, length_control.md)*

Association with `tokens_used` (Spearman)	Unadjusted	Controlling for record count
File size	0.931	−0.019
Compacted-record bytes	0.869	0.133
Record count	0.960	—

- **[O]** File size and token totals are strongly associated, but record count, a proxy for session length, is the strongest correlate. After partialling out record count, file size has no association with tokens (−0.019) and compacted bytes only a weak one (0.133). Bytes per record is unrelated to tokens per record (−0.009).
- **[O]** Tokens per record correlate with the share of compacted bytes (0.700) and with the number of compactions per 1,000 records (0.697).
- **[I]** The most plausible reading is reverse direction: compaction is triggered when the context approaches the model window, so context-heavy sessions both consume more tokens per step and compact more often. A causal effect of stored compaction bytes on tokens is not supported by these data. Within-stratum comparisons were possible for only 48 files and are inconclusive.
- **[O]** Subagent sessions are smaller than their parents. The child had fewer tokens in 39 of 41 linked pairs and a smaller file in 40 of 41. The median child had 3,757,291 tokens against 75,380,819 for the median parent, so these children do not appear to inherit the parent's full token footprint.

10.5 Compaction and input size

- **[O]** In the reconstructed turn, ordinary input fell from 230,382 tokens immediately before compaction to 34,868 immediately after. Meanwhile the compacted record written to disk occupied 1,137,409 bytes, and the compaction response itself consumed 230,614 input and 2,826 output tokens. *(turn_token_timeline_v1)*
- **[I]** Context compaction reduces the model's working context while increasing persistent storage. The two are distinct operations with opposite size effects.

10.6 Repeated usage records in the logs

- **[O]** The corpus yields 1,778,473 token-usage observations. A rule-based check flags 481,420 of them (27.1%) as possible duplicates, in 64,454 groups. The duplicates take two forms:
  - **Same response.** 16,353 rows repeat an earlier response: same response ID, scope and six counters.
  - **Same snapshot.** 465,067 rows repeat a "last usage" snapshot that is identical, together with its cumulative companion counters.

  (token_usage_duplicate_candidates_v1)
- **[I]** Summing usage rows from the logs would therefore overcount. Flags mark candidates, not proven duplicate charges, and no row was removed.
- **[O]** Across six diagnostic responses, the reasoning share of output tokens ranged from 0% to 64.9%. The logs contain no counters below the output total, so the split among reply text, tool calls and internal

formatting is unknown. *(output_token_components_v1)*

Limitations

- The request-level evidence is a case study: 3 log conversations linked to the corpus, 21 paired requests, 14–18 June, with no explicit request-to-response identifiers.
- The file-level analysis covers files through 18 June only. Files linked by forking or spawning may share history and are not fully independent.
- No server-side tokenizer, per-item token attribution or billing data were available. No monetary estimates are made.
- Paired request modes involve different models and contexts. No efficiency ranking of incremental versus full sends is justified.

Implications

- **[P]** Storage optimizations, such as content-addressed images and non-recursive compaction snapshots, can be evaluated mainly on disk, memory and load-time grounds. Their token effect is indirect.
- **[P]** Preserving continuation state (stable request properties, connection reuse, resume warmup) is the lever that avoids full-history resends, including embedded images. The frequency of such resends in production remains to be measured.
- **[P]** Persisting a compact per-request record of input composition and usage, keyed by `response_id`, would make per-content token attribution possible without storing content.

11. Duplication and Subagent Amplification

This section covers RQ7 (duplication) and RQ8 (subagent and parent amplification). Labels and paths follow Section 7. "Duplicate" means an exact byte or hash match; semantic duplication was not measured.

11.1 Images in compacted histories

[O] Every embedded image in `replacement_history` and `guardian_history` was decoded and hashed (*compacted_image_duplicates_v1*):

Measure	Value
Image appearances	99,244
Distinct images	1,936
Decoded size of distinct images	0.61 GiB
Stored as base64 JSON	38.03 GiB
Duplicate share of stored image bytes	97.85%

The duplicate bytes fall into disjoint categories:

Category	Appearances	Stored bytes
Same image in a later compacted record of the same file and history	91,571	38,045,175,558
Repeated within the same record	3,785	1,068,305,807
Same image in another file	1,747	752,086,625
Same image in both histories of a record	205	93,794,408
First copy retained	1,936	879,067,554

[O] Persistence. Of 3,047 image groups that recur within a file and history, 3,042 appear in every compacted record between their first and last appearance. Once captured, an image is carried through every subsequent snapshot. One image recurred in 178 consecutive compacted records.

11.2 Text in compacted histories

[O] The text strings in compacted histories (content text, reasoning summaries, tool inputs and outputs) were hashed (*compacted_text_duplicates_v1*):

Measure	Value
Occurrences	1,899,050
Distinct blocks	181,206
Distinct text, UTF-8	0.39 GiB
Stored	1.87 GiB
Duplicates	1.46 GiB (78.04%)
Of which repeated across compacted records of the same file	1.23 GiB
Of which repeated across files	0.11 GiB

In `replacement_history` alone, 1.32 GiB of stored text reduces to 41,969 distinct blocks (0.063 GiB of UTF-8).

[I] Text repeats in the same way as images. It follows the same carry-forward pattern but is about 20 times smaller in volume. Removing image repetition is therefore what dominates any storage reduction (Section 14).

11.3 Records repeated by resume and forks

[O] Repeated metadata. The corpus holds 10,562 `session_meta` records for 845 files (*record_type_counts_corrected_v1.json*). In the largest file, Codex's October rewrite reduced 439 `session_meta` records to 1 and removed 4,870 `event_msg` and 488 `response_item` records that did not reappear in the rewritten file (*zstd_comparison/stage2_summary.json*).

[I] Replay on resume. Before October 2026, resuming or forking a session re-wrote earlier records into the same file. Physical record counts therefore overstate distinct messages and actions.

11.4 Subagents

[O] Spawn relationships. Codex's local state database records 151 parent-to-child spawn edges; 110 threads are labelled as subagent threads (*SQLITE_SCHEMA.md*, `thread_spawn_edges`, `threads`).

[O] One measured case. A subagent rollout of 1,048,636,822 bytes, containing 138 compacted records, is 8,474,017 bytes after Codex's October rewrite:

- its records fell from about 32,000 to 258;
- it gained a `subagent_history_start_ordinal` field that points into the parent's history instead of copying it.

10 of its 23 distinct images do not appear in the rewritten file. Whether they are present in the parent was not checked. *(zstd_comparison/stage1_summary.json, stage2_summary.json)*

[O] Similar large files. Among the ten largest corpus files, seven are nearly identical in size and structure:

- 0.92–0.98 GiB each, with 130–138 compacted records;
- the same first, last and maximum snapshot sizes (0.075, about 7.37 and 9.15 MiB).

(compaction_growth_v1)

[H] Copied parent history. These files are consistent with subagent or forked sessions that each carry a copy of the same parent history, as in the measured case. This was not verified file by file.

11.5 Summary

- **[O] Images.** Image duplication dominates: 97.85% of stored image bytes in compacted histories are repeats, mostly carried forward within a session (35.43 GiB) rather than shared across files (0.70 GiB).
- **[O] Text.** Text duplication follows the same pattern at 1.46 GiB.
- **[O] Subagents.** In one measured case, a subagent file was reduced 124× by referencing its parent's history instead of copying it. Seven of the ten largest files show the same size signature.
- **[I] Implication.** Storing content once and referencing it, by content hash for images and items and by ordinal for inherited history, addresses all three sources of duplication.

Limits. Matching is by exact hash, so re-encoded or resized images count as distinct. The cross-file subagent hypothesis rests on one verified case and on size signatures.

12. Before and After: Codex's October 2026 Storage Changes

During this study, the public Codex client `rust-v0.164.0` (commit `6aaac69`, released 2026-10-23) changed how rollouts are stored. Because our corpus was copied before the change, we could compare the same sessions before and after. Labels and paths follow Section 7.

12.1 What changed (public code)

[O] Background compression (`codex-rs/rollout/src/compression.rs`):

- Rollouts not modified for 7 days are compressed with zstd level 3 and get a `.zst` suffix.
- The plain file is deleted once the compressed copy is verified.
- A worker runs at startup and on RPC, with at most 2 concurrent jobs and a 5-hour budget.
- Appending to a compressed session first decompresses the whole file (`materialize_rollout_for_append`).

[O] Legacy rollout rewrite. Observed by comparing files; the responsible code was not located:

- every record gains a consecutive `ordinal`;
- repeated `session_meta` records are removed;
- superseded compacted snapshots are pruned;
- subagent rollouts reference the parent history by ordinal.

12.2 Whole corpus

[O] We matched files by name in the local Codex store; no file was opened (*zst_counterpart_sizes_v1*):

Measure	Value
Corpus files with a `.zst` counterpart	843 of 845
Original bytes of these files	56,353,330,276 (52.48 GiB)
Codex `.zst` bytes	11,381,948,312 (10.60 GiB)
Overall reduction	4.95×

12.3 Three measured pairs

[O] For the three largest corpus files, we decompressed copies of the Codex `.zst` and compared them, record by record, with our pre-change copies. Both copies of each pair had identical modification times (*zstd_comparison/stage1_summary.json, stage2_summary.json*):

Pair	Before	After (decompressed)	`.zst`	What changed
pair_01, the largest file	8,528,749,901	307,085,723	160,663,901	163 of 190 snapshots removed; all 220 distinct images kept; image appearances 15,269 → 639; 439 → 1 `session_meta`
pair_02	1,189,302,022	1,196,926,223	775,005,614	all 134 snapshots and 2,496 image appearances kept; only ordinals added; zstd ratio 1.53×
pair_03, a subagent	1,048,636,822	8,474,017	5,725,256	about 32,000 records → 258; history referenced from the parent; 10 of 23 distinct images absent

12.4 Interpretation

- **[I] What the rewrite removes.** It removes redundancy left by resume replay and by inherited subagent history. Where a session's snapshots were all distinct windows (pair_02), it removes nothing, and compression alone yields 1.53×.
- **[I] Why compression cannot remove image repetition.** At level 3, zstd's window is 2 MiB (`windowLog` 21 in zstd's `clevels.h`), shorter than the 7–9 MB snapshots in which an image recurs. Each copy is therefore compressed again rather than referenced.
- **[O] New snapshots are unchanged.** Snapshots written after the change still embed images. In four sessions active between 2026-10-16 and 2026-10-28, every compaction wrote a full `replacement_history` with images (*post_october_compaction_sample_v1*).
- **[O] Tokens are not affected.** Token usage follows the context sent to the server, not file size. After controlling for session length, file size is not associated with `tokens_used` (partial Spearman −0.019, n = 189; *tokens_vs_storage_v1/length_control.md*).

12.5 Summary

[O] Codex's October changes cut the stored corpus from 52.48 to 10.60 GiB, through pruning, subagent references and zstd compression.

[I] They leave the underlying pattern in place: surviving and new snapshots still re-store the same images and retained messages. They also add a full-decompression step before appending.

Section 14 measures how far references, rather than compression, go on the same files.

Limits.

- Modification times of the 843 `.zst` files were not compared with the corpus copies. Sessions continued after our copy would be larger in the Codex store.
- The rewrite code was not located, so its rules are inferred from three pairs.

13. Proposed Architecture: A Reference-Based, Layered Session Store

Status: **[P] proposal**, designed by the researcher; its storage effect is evaluated in Section 14, other effects remain to be measured. Supporting figures cite the reports they come from.

13.1 Problem it addresses

- **[O]** Before October 2026, each compaction wrote a complete `replacement_history` snapshot, although about 98% of it was identical to the previous snapshot. In three consecutive snapshots, 270–271 items were kept unchanged and only 6–8 were new. *(zstd_comparison/compaction_snapshot_sample.md)*
- **[O]** Images repeated from one snapshot to the next account for 36.39 GiB, or 89.91% of compacted bytes. The corpus contains only 1,936 unique images (0.61 GiB). *(compaction_growth_v1/text_growth.md; compacted_image_duplicates_v1)*
- **[O]** In October 2026 Codex began referencing parent history from subagent rollouts by ordinal, instead of copying it (`subagent_history_start_ordinal`). Every record now carries an `ordinal`. Compaction snapshots are not yet stored this way. *(zstd_comparison/stage2_summary.json)*

13.2 The idea

1. **Separate context store.** Context items live in a store outside the session log: messages, tool results, images, summaries and decisions. Each item has a stable identifier and a content hash.
2. **Compaction as a reference.** The session log records a compaction event that points to the items forming the new context. It does not copy them.
3. **Append-only.** Each compaction adds only what is new, such as new messages and the new summary. Earlier items and summaries are kept, not rewritten or deleted.
4. **Versioned, addressable segments.** Items are grouped into ordered, versioned segments with boundaries. A consumer can request a range ("items X to Y") or a specific version, rather than the whole history.
5. **Partial, on-demand loading.** The agent loads only what the current step needs. Older segments are available for retrieval, not loaded by default.
6. **Later refinement.** Pruning and summarization of old segments can be added and improved without changing the log format.
7. **Side layers.** A decision layer and a memory layer record approved decisions, constraints and facts. They answer most continuity needs, so full history is loaded less often.

13.2.1 Design refinements

1. **[P] Incremental compaction.** Compaction appends only what is new to the existing summary and references, instead of re-summarizing the whole context.
   - **Rationale.** The observed compaction call processed the full context: 230,614 input tokens in the reconstructed turn (*turn_token_timeline_v1*). An incremental step processes only the new part, so its input is smaller by construction.
   - **To be measured.** The size of the time and token reduction.
2. **[I] Snapshot as a reference list.** A compacted snapshot becomes a list of item references. On average a snapshot holds 146 items (1,224,192 items in 8,363 records).
   - With references of about 12 bytes, that is roughly 2 KB per snapshot, against 9.1 MB in the sampled snapshots (*compacted_item_sizes_v1*, *compaction_snapshot_sample.md*).
3. **[P] Window-aware loading.** Segments are loaded up to the capacity of the selected model's window. A switch to a smaller-window model then selects fewer segments, rather than running a `ModelDownshift` compaction of the whole history (`session/turn.rs`). The same store can hold more history than any single window, and the loaded part can be chosen by relevance.
4. **[P] Compression only above a threshold.** Session files remain plain JSONL, with images and repeated items held by reference. File compression applies only when a session exceeds a configurable size.
5. **[P] Segmented compression.** Compressed sessions are stored as independent frames or segments. Appending then writes a new segment, instead of decompressing and rewriting the whole file

(`materialize_rollout_for_append` in `compression.rs`).

6. **[P] Media store and deletion.**
   - **Storage.** Images live in a SQLite `media` table, linked to sessions by `session_media`. A binary BLOB is about 25% smaller than base64 text for the same image.
   - **Deletion.** When a user removes an image, its references are kept and marked as removed by the user, so resume reports the removal explicitly instead of failing.
   - **Recovery.** Missing media can be restored from a backup location when loss is detected.
   - **Protection.** Renaming files or hiding the file type is obfuscation only. Protecting sensitive images requires encryption at rest.
7. **[P] Compaction snapshots stored apart from the session log.** Snapshots live in their own store, and the session log keeps a reference to the current snapshot.
   - **[O] Size.** Compacted records are 77.13% of corpus bytes, while user-role and assistant messages are 2.89% (*storage_composition_v1*).
   - **[I] Only the latest snapshot is needed to resume**, because each snapshot replaces the history before it. Codex's own October rewrite pruned superseded snapshots, 190 to 27 in one file (*zstd_comparison/stage2_summary.json*).
   - **Consequence.** Older snapshots can be archived or pruned by policy, and the log can be read without parsing any snapshot.
8. **[P] Project-level side memory file, built by extraction.** An extraction tool runs after each context window. It records decisions and constraints, working state, project facts and an index of significant tool outcomes, each entry linked to its source item.
   - **Role.** The file is a side memory at project level. It is **not a replacement** for the context or for compaction: it supplements them, carries continuity across sessions of the same project, and lets resume start from a compact, verifiable record.
   - **Design constraints.**
     - **[O] Extraction must be incremental.** Codex's own memory generation failed in 4 of 20 jobs because the model context window was exhausted (*SQLITE_SCHEMA.md*, memories tables).
     - **[O] Extraction must work from readable items.** The compaction summary is stored as encrypted content and cannot be read locally (*compacted_item_sizes_v1*).
   - **Risks.** Extraction errors, which source links make verifiable; the token cost of extraction; stale or conflicting entries, which need supersession rules; and secrets collected into memory, which need exclusion and deletion rules.

13.3 Expected effects (to be measured)

Aspect	Current behaviour (pre-October data)	Expected with the proposal
Storage per compaction	full snapshot (e.g. 9.1 MB, ~98% repeated)	new items only (~2% of a snapshot in the sample)
Total storage, measured on 843 files	Codex `.zst` after the October rewrite: 10.60 GiB	images by reference with a shared media table, zstd-3: 3.00 GiB (3.53× smaller) *(media_reference_comparison_v2)*
Images	re-serialized into every snapshot	stored once, referenced
Resume	parse full snapshots, base64 included	load the latest checkpoint plus needed segments
Tokens	full context assembled for each request	unchanged unless partial loading and the decision layer reduce what is sent

[I] Storage, RAM and resume time are the direct gains. Token savings come only from sending less, through partial loading and the decision and memory layers, not from the storage format itself. This is consistent with the absence of any size–token association once session length is controlled. *(tokens_vs_storage_v1/length_control.md)*

13.4 Design questions

- **Reconstruction cost.** A long chain of deltas is slow to replay. Use periodic full checkpoints, every N compactions or every M items, with deltas between them.
- **What is sent to the model.** The assembled request is still a full context. The proposal must define how the agent requests segments: a retrieval tool, automatic selection by relevance, or both.
- **Segment index.** Each segment needs a short descriptor (type, turn range, hash, size, summary) so that the right range can be selected without loading it.
- **Reference integrity.** Use reference counting before deleting items. Exports and archives must bundle referenced items, and resume must detect missing references.
- **Prompt caching.** Keep a stable prefix so that incremental requests and caching continue to work after compaction.
- **Privacy and retention.** A separate item store allows per-item deletion and redaction, for example of images or secrets, without rewriting session logs.

13.5 Evaluation plan

1. Simulate the store on corpus copies and measure total bytes, checkpoint and delta sizes, and reconstruction time against the current format.
2. Measure resume RAM and latency for the current format, the October format and the proposal.
3. In a controlled experiment, measure tokens with partial loading and a decision layer compared with full-context requests.

14. Quantitative Evaluation: References versus Compression

We measured how much of the redundancy described in Sections 7–12 is removed when content is stored once and referenced, and compared the result with Codex's own post-October format on the same files. Labels and paths follow Section 7.

14.1 Variants

All variants are compressed with zstd level 3, the same as Codex's.

Variant	Construction
Theirs	Codex's actual `.jsonl.zst`: legacy rewrite plus zstd-3
Ours v1	Each embedded data-URL image is replaced by a short reference `@@media:<n>`, and the file is compressed with zstd-3. Each distinct image is stored once at decoded size: either per session, or in a media table shared across sessions with a `session_media` link table
Ours v2	Ours v1, plus each item of a compacted record's `replacement_history` and `guardian_history` is replaced by `"@@item:<n>"`. Each distinct item is stored once per session in a zstd-3 item store, so a snapshot becomes a list of references

Method.

- Each source file was read once, read-only.
- All outputs were compressed in memory, and only byte counts were kept.
- The shared media and link tables were built in SQLite from hashes and sizes, and measured.

- Synthetic tests verify byte-exact reconstruction of the original from the main stream, the item store and the media map, at chunk sizes from 1 byte to 1 MiB.

(scripts `compare_media_reference.py`, `compare_media_reference_v2.py`)

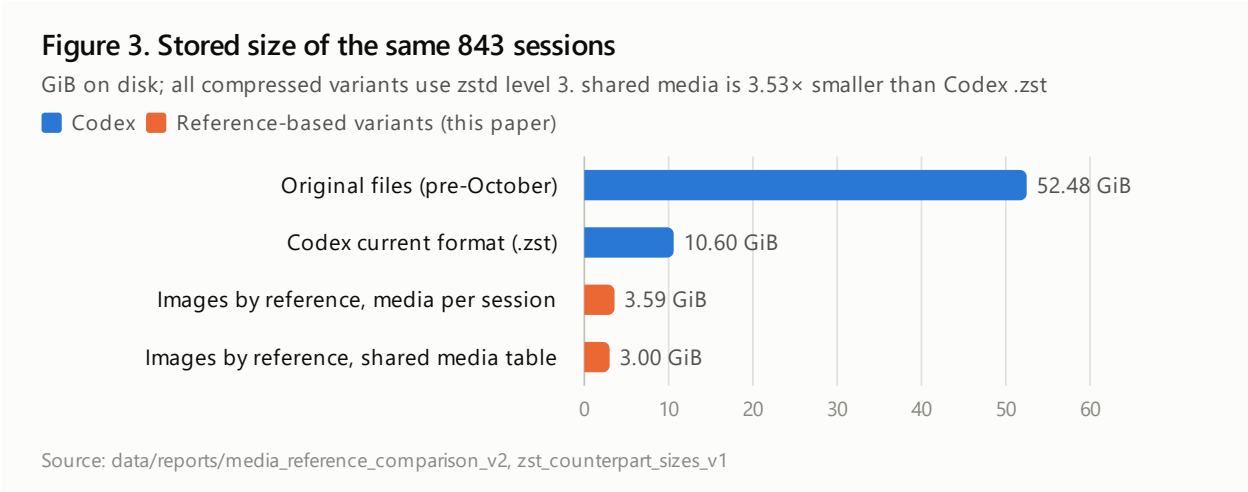
14.2 Whole corpus: 843 files with a Codex `.zst`

(media_reference_comparison_v2/summary_corpus.json)

Version	Bytes	GiB	Theirs ÷ version
Original, pre-October	56,353,330,276	52.48	0.20×
Theirs (Codex `.zst`)	**11,381,948,312**	**10.60**	**1.00×**
Ours v1, media per session	3,853,059,232	3.59	2.95×
Ours v1, shared media table	**3,225,189,819**	**3.00**	**3.53×**
Ours v2, media per session	3,852,692,848	3.59	2.95×
Ours v2, shared media table	3,224,823,435	3.00	3.53×

Components:

Component	Bytes
v1 file, zstd-3	1,840,652,475
Distinct images, shared (3,969 images)	1,384,078,592
Per-session copies of images (6,368 file-image pairs)	2,012,406,757
SQLite `media` and `session_media` tables	458,752



[0] Validation. The run reproduces earlier reports exactly:

- 8,363 compacted records;
- 1,424,366 history items, which equals 1,224,192 in `replacement_history` plus 200,174 in `guardian_history`;
- 99,244 images inside those items.

There were 0 marker collisions and 0 truncated records.

14.3 Six post-October files

The two largest corpus files in their rewritten form, plus four recent sessions (*media_reference_comparison_v1, v2/summary_study.json*):

File	Theirs	Ours v1	Theirs ÷ ours v1
pair_01	160.7 MB	95.2 MB	1.69×
pair_02	775.0 MB	69.5 MB	11.15×
Session 2026-10-16	32.8 MB	6.8 MB	4.84×
Session 2026-10-20 (a)	36.0 MB	7.7 MB	4.66×
Session 2026-10-20 (b)	33.1 MB	21.6 MB	1.53×
Session 2026-10-22	20.8 MB	15.6 MB	1.34×
Total	**1,058.4 MB**	**216.4 MB**	**4.89×**

For the recent sessions, Codex had not compressed them yet; "theirs" is zstd-3 of the file as written.

[O] Calibration. Our zstd-3 of the decompressed pair_01 and pair_02 reproduced the size of Codex's own `.zst` byte for byte.

14.4 Findings

- **[O] Corpus-wide.** Referencing images from a shared media table stores the corpus in 3.00 GiB, against 10.60 GiB in Codex's current format (3.53× smaller) and 52.48 GiB originally (17.5× smaller).
- **[O] Shared media.** A media table shared across sessions saves 628 MB over per-session media: 3,969 distinct images instead of 6,368 file-image pairs.
- **[O] Item references.** These shrink the uncompressed size by 1.28 GB, but after zstd-3 they change the total by only 0.37 MB.
  - **[I] Why.** Once images are removed, consecutive snapshots are small enough (largest item 238 KB) for zstd's 2 MiB window to remove repeated text itself. Their value lies in uncompressed storage and in parsing and memory at resume, which we did not measure.
- **[O] Where the gain is.** The gain is concentrated in long sessions with images. In the ten largest files, theirs is 1.507 GB and ours v1 is 0.337 GB. The per-file median ratio is 1.00 (P10 0.74, P90 1.47).
- **[O] Where Codex is smaller.** In 199 mostly small files, Codex's format is smaller: 534.5 MB against 803.7 MB.
- **[H] Why Codex is smaller there.** The legacy rewrite removes replayed records and replaces inherited subagent history with a parent reference (Section 12). Our variants do neither.
- **[I] The approaches are complementary.**

Source of redundancy	Addressed by
Images re-stored within a session	Content-addressed media with references
Repeated or inherited records	Codex's pruning and parent references
Residual repetition	Compression

  The combination, not any single mechanism, is the efficient design (Section 13).

14.5 Limits

- Storage bytes only: these variants change what is written to disk, not what is sent to the model, so no token effect is claimed.
- Media is counted at decoded size, without filesystem overhead.
- Item identity is exact bytes.
- File modification times of the Codex `.zst` files were not compared with the corpus copies.
- Resume memory and latency were not measured for any variant.

15. Security and Privacy Implications

This section covers RQ10. We describe what the local persistence model exposes, by category and volume. No content was inspected to establish it. Labels and paths follow Section 7.

15.1 What is stored locally

Surface	Evidence	Exposure
Embedded images	**[O]** 106,260 data-URL occurrences of 3,969 distinct images (*media_reference_comparison_v2*)	Screenshots of screens, code, documents and dashboards, stored as plain base64
Tool outputs	**[O]** 3.73 GB in 463,596 records; 926 outputs exceed 65,536 characters (*response_item_profile_v1*)	Command output, file contents, logs, error traces, and any secret they print
Injected context	**[O]** The `user` role also carries client-injected context, such as project instructions and environment descriptions (*compacted_profile_v1*)	Project rules, paths and environment details
Reasoning	**[O]** All 383,510 reasoning records carry encrypted content; 130,884 also carry readable summary text	Readable summaries of the agent's reasoning
Trace logs in the local state database	**[O]** 4,515 of 10,610 log rows are at TRACE level, the largest body is 4.35 MB, and a case study found 27 structured model requests with 9 embedded image occurrences (*SQLITE_SCHEMA.md*, *logs_request_capture_v1*)	Request bodies, as sent to the model, in a second store beside the rollouts
Thread catalog, goals and memories	**[O]** 449 catalog rows, 314 threads, 13 memory outputs (*SQLITE_SCHEMA.md*)	Titles, workspace references and generated memories

[O] Compression is not protection. Since October 2026, idle rollouts are compressed with zstd, which any user can decompress (Section 12). The compression changes the format, not the exposure.

15.2 Repetition multiplies exposure

- **[O] Carry-forward.** An image captured once is re-stored in every later snapshot: 3,042 of 3,047 recurring image groups appear in every snapshot between their first and last appearance, and one image recurred in 178 consecutive snapshots (*compacted_image_duplicates_v1*).
- **[O] Copies across files.** Subagent and forked files can carry a copy of the parent history, as in the 1.05 GB subagent file of Section 11.4.
- **[I] Deletion is costly.** Removing one sensitive screenshot from local history therefore means finding and rewriting every snapshot, and every file, that carries it. In a content-addressed design the image exists once and can be removed, or replaced by a tombstone, in one place (Section 13).

15.3 Design implications

- **[P] One copy, with an explicit tombstone.** Store media and large outputs once and reference them. When a user removes an item, keep its references with a marker showing that the user removed it, so that resume reports the gap instead of failing.
- **[P] Encryption at rest for sensitive content.** Renaming files or obscuring their type is obfuscation, not protection.
- **[P] Bounded trace logging.** Exclude or redact request bodies from default trace retention, and apply size and age limits to log rows.
- **[P] Retention controls.** Expose per-session and per-item retention, export and deletion to the user, together with a view of what is stored, per Section 7's categories.

Limits. Exposure is described by structure and volume. We did not inspect content, so we make no claim about whether specific secrets are present.

16. Limitations

16.1 Data

- **One user.** The corpus comes from one heavy user on one machine, with several clients and model generations. Shares and ratios describe this workload and are not population estimates.
- **Period.** The corpus predates Codex's October 2026 storage changes. Post-change behaviour is measured on three pairs, four recent sessions and the sizes of 843 `.zst` files.
- **Files and sessions.** Files are not sessions. Forks, resumes and subagents produce related files, and physical record counts include replayed records.
- **Modification times.** The 843 Codex `.zst` files were matched by name only, without comparing modification times. Sessions continued after our copy would make Codex's sizes larger, and so overstate our advantage in Section 14.

16.2 Measurement

- **Exact matching only.** Duplication is exact byte or hash identity. Semantically equivalent, re-encoded or resized content counts as distinct, so duplication is a lower bound.
- **Bytes, not tokens.** No storage share is converted into tokens or cost.
- **Token evidence.** It rests on recorded counters and diagnostic cases:
  - one fully reconstructed turn;
  - 21 paired requests from local trace logs;
  - 74 snapshot-attached usage readings.

  No server-side tokenizer, per-item attribution or billing data were available.
- **Durations.** They rely on client timestamps, present for 755 of 1,314 compaction events.
- **Privacy by design.** Tool names, arguments and content were not exported. Results are therefore broken down by record subtype, not by tool or by content kind; pasted text, for example, is not separated from typed text.

16.3 Inference

- **Compaction path.** The path used in practice, local or remote, was not observed. It is inferred from code and from images retained in recent snapshots.
- **Resume.** Its requirements are derived from code and structure, not from controlled resume experiments.
- **Legacy rewrite.** Its rules are inferred from three before/after pairs; the code responsible was not located.
- **Subagents.** Corpus-wide subagent duplication is hypothesized from one measured case and size signatures.

16.4 Not measured

Item	Where it matters
Resume memory and latency, for any format	Sections 9.5, 14
Equivalence of context reconstructed from a rewritten rollout	Section 12
Token effect of sending images by reference or loading them on demand	Section 13
Time and token effect of incremental compaction and of a project memory file	Section 13
Presence in the parent file of the 10 images missing from the rewritten subagent file	Section 11.4

These are the measurements a follow-up study, or OpenAI with server-side data, should perform first.

17. Recommendations for OpenAI

The recommendations are grouped by area. Each one states the evidence behind it, and whether its effect has been measured here (**M**), is expected from the mechanism (**E**), or needs a controlled experiment (**X**). All are **[P]** proposals. Section numbers refer to the evidence.

17.1 Storage

1. **Content-addressed media, shared across sessions (M).** Store each image once in a media table, link it to sessions through a link table, and keep only a short reference in rollouts and snapshots.
   - On 843 files this stores 3.00 GiB, against 10.60 GiB in the current format: 3.53× smaller. Sharing across sessions saves a further 628 MB (Section 14).
2. **Snapshots by reference, in a separate store (M/E).** Persist each compaction snapshot as a list of item references, or as changes from the previous snapshot, in a store separate from the session log.
   - Keep the latest snapshot for resume, and prune or archive older ones by policy.
   - Consecutive snapshots are about 98% identical, and 89.91% of compacted bytes are images carried from the preceding snapshot (Section 9.2). Codex's own rewrite already prunes superseded snapshots (Section 12).
3. **Reference inherited history everywhere (M).** Keep the October parent reference for subagents and extend it to forks and resumes. In one case it reduced a subagent file 124× (Section 11.4).
   - Validate references when a parent is archived, moved or deleted, and report a missing parent explicitly on resume.
4. **Segmented compression and append without rewrite (E).** Store compressed sessions as independent frames, so that appending writes a new segment instead of decompressing the whole file (`materialize_rollout_for_append`, Section 9.5).
   - Compress only above a configurable size, once images and repeated items are held by reference.

17.2 Compaction and context

1. **Incremental compaction (E/X).** Extend the existing summary with what is new, instead of re-summarizing the full context.
   - Today compaction is a full-context call: 230,614 input tokens in the reconstructed turn, and a median of 56.5 s (Section 9.3). An incremental step processes less input by construction. The size of the gain needs measurement.
2. **Prefix-stable compaction (X).** Design the replacement history to keep a stable prefix, so that the next request can stay incremental and cached instead of being sent in full (Section 10).
3. **Image lifecycle in context (X).** Keep an image in the active context only while it is in use. Afterwards, replace it with a short description and a reference that can be re-attached on demand.
   - The local path drops images entirely. The remote path keeps them unless an image budget is enabled, and recent snapshots still carry them (Section 9.4).
4. **Window-aware partial loading (E/X).** Load stored segments up to the selected model's window. A switch to a smaller model then selects fewer segments, instead of compacting the whole history (`ModelDownshift`, Section 9.4).

17.3 Request continuity

1. **Protect incremental requests (X).** The client already computes why a request is sent in full.
   - Avoid mid-turn changes to model or tools, reuse connections, and warm continuation state on resume.

- Expose the full-resend reason in telemetry, to show how often full resends occur.

17.4 Accounting transparency

1. **Separate cached and uncached input (M).** `tokens_used` equals the cumulative total including cached input, so a single total overstates newly processed input (Section 10).
2. **Persist a per-request manifest (X).** For each request, record item types, sizes and content hashes, keyed by `response_id`. This makes per-content token attribution possible without storing content; current logs cannot support it.
3. **Log each usage once, and align event counters (M).**
   - 27.1% of logged usage rows are candidate duplicates (Section 10).
   - In the reconstructed turn, the event-stream cumulative total omitted the compaction response's own usage.

17.5 Memory

1. **A project-level side memory file (X).** Run an incremental extraction after each context window. It records decisions, constraints, working state, project facts and an index of significant tool outcomes, each entry linked to its source item.
   - The file complements the context and compaction; it does not replace them.
   - Extraction must be incremental, because 4 of 20 memory jobs failed when the context window was exhausted.
   - It must work from readable items, because compaction summaries are encrypted (Sections 9.1, 13).

17.6 Privacy

1. **Single-copy deletion, tombstones, encryption and bounded trace logs (E).**
   - Store sensitive content once, so that one deletion removes it, and leave a "removed by user" marker for resume.
   - Encrypt sensitive content at rest.
   - Exclude request bodies from default trace retention, where TRACE rows reach 4.35 MB (Section 15).

17.7 Priorities

Priority	Recommendations	Reason
Highest	1, 2, 3	Measured on real files, largest storage effect, and consistent with the direction of Codex's October changes
Next	5, 6, 13	Address time, tokens and continuity, where storage changes alone have no effect (Section 10)
Then	10–12, 14	Low cost, and improve trust in usage reporting and local privacy

18. Conclusion

We measured every stored byte in 52.49 GiB of real Codex sessions without reading their content. The record that lets a coding agent resume long work is dominated by neither the conversation nor the work itself:

- the conversation is 2.89% of stored bytes;
- execution traces are about 15%;
- 77.13% are compaction snapshots, which re-store the retained history in full at every window;
- 69.34% of the corpus consists of images already present in the preceding snapshot.

Compaction reduces what the model sees and, at the same time, increases what is kept on disk. Its cost in time and tokens comes from summarizing the full context. The cost of what it writes comes from copying instead of referencing.

Codex's October 2026 changes reduced the same files from 52.48 to 10.60 GiB by pruning superseded snapshots, referencing parent history from subagents and compressing. Snapshots written after the change still embed their images, and compression cannot reach repetition spread across multi-megabyte snapshots. Storing each image once and referencing it, from a media table shared across sessions, reduces the same files to 3.00 GiB, 3.53× below the current format. The two approaches remove different redundancy and work best together.

Storage is the easiest gain to measure, but not the only one. The same principle of storing once and referencing applies to snapshots, to inherited history and to compaction itself. It also points to a session architecture with separate layers:

- a conversation log;
- a snapshot store holding only the latest checkpoint;
- a shared media store;
- a project-level memory file built incrementally with links to its sources.

Such a design could make resume lighter and compaction cheaper, and could let a smaller model load what fits instead of compacting again. These effects are expected from the mechanism but not yet measured. We list the experiments that would establish them (Section 16.4) and the changes we recommend (Section 17).

19. Appendices

Appendix A. Analyzers and their reports

All scripts are in `scripts/`, use the Python standard library only, and have synthetic tests (`scripts/test_*.py`, 27 test modules). Commands, limits and caveats are documented in `scripts/README.md`. Reports are in `data/reports/` unless stated.

Stage	Scripts	Reports
Inventory and survey	`survey_sessions.py`, `fast_session_index.py`, `import_session_survey.py`, `import_session_index.py`	`DATASET_INVENTORY.md`
Record index	`index_session_records.py`, `repair_null_record_types.py`, `export_session_statistics.py`, `resolve_compound_session_ids.py`, `apply_compound_session_ids.py`	`record_type_counts_corrected_v1.json`, `null_record_type_repair_v1.json`
Structural censuses	`profile_record_shapes.py`, `profile_response_items.py`, `profile_compacted.py`	`response_item_profile_v1`, `event_msg_profile_v1`, `compacted_profile_v1`, `compacted_item_sizes_v1`, `STRUCTURAL_ANALYSIS_STEPS_1_3.md`
Hash indexes	`profile_compacted_images.py`, `profile_compacted_texts.py`	`compacted_image_duplicates_v1`, `compacted_text_duplicates_v1`
Growth and composition	`analyze_compaction_growth.py`, `analyze_compaction_text_growth.py`, `analyze_storage_composition.py`, `analyze_storage_composition_event_projection.py`	`compaction_growth_v1`, `storage_composition_v1`
Token observations	`index_token_usage.py`, `token_usage_stream.py`, `mark_token_usage_duplicates.py`	`token_usage_observations_v1`, `token_usage_duplicate_candidates_v1`

Stage	Scripts	Reports
Token case studies	`analyze_session_storage_tokens.py`, `analyze_output_token_components.py`, `analyze_turn_token_timeline.py`, `audit_model_request_capture.py`, `analyze_logs_request_capture.py`, `analyze_logs_request_comparison.py`, `analyze_tokens_vs_storage.py`, `analyze_length_control.py`, `analyze_compaction_cost.py`	`storage_token_case_study_v1`, `output_token_components_v1`, `turn_token_timeline_v1`, `model_request_capture_audit_v1`, `logs_request_capture_v1`, `tokens_vs_storage_v1`, `compaction_cost_v1`
Local state	`export_sqlite_tables.py`, `import_codex_sqlite.py`	`SQLITE_SCHEMA.md`, `sqlite_metadata_import_v1`
Before/after	`compare_zstd_pairs.py`, `compare_zstd_stage2.py`, `list_zst_counterparts.py`	`zstd_comparison/stage1_summary.json`, `stage2_summary.json`, `zst_counterpart_sizes_v1`
Reference variants	`compare_media_reference.py`, `compare_media_reference_v2.py`	`media_reference_comparison_v1`, `media_reference_comparison_v2`

Appendix B. Research database

PostgreSQL 18, created by migrations `database/migrations/001`–`006`. No table stores conversation text, images or commands.

Table	Rows	Content
`research_sessions`	845	one row per file: size, record count, scan state
`research_session_records`	3,497,535	one locator per physical record: file, line, byte offset, byte length, timestamp text, top-level type
`research_token_usage_observations`	1,778,473	each token-counter object: locator, identifiers, six counters, duplicate-candidate flag
Imported SQLite metadata (9 tables, prefixed `state5_`, `codexdev_`, `memories1_`, `logs2_`)	11,714	metadata from Codex's local stores, with text replaced by lengths and hashes

Appendix C. Reproduction

Each report folder contains a `summary.json` with exact totals, the analyzer version, runtime and peak working set. Each analyzer:

- defaults to a read-only plan;
- writes to a new versioned folder;
- resumes from per-file checkpoints.

The corpus-wide reference comparison of Section 14 is reproduced with:

```
python -B scripts/list_zst_counterparts.py
python -B scripts/compare_media_reference_v2.py --set corpus
python -B scripts/compare_media_reference_v2.py --set study
```

It requires Python 3.14 or later, for `compression.zstd`.

Appendix D. Public code references

The `openai/codex` repository, tag `rust-v0.164.0`, commit `6aaac694803004a63c6f393d153c96d22521a6b9`:

File	Relevant content
`codex-rs/core/src/session/turn.rs`	compaction triggers (pre-turn, mid-turn, post-turn, `ModelDownshift`) and path selection
`codex-rs/core/src/compact.rs`	local compaction: `COMPACT_USER_MESSAGE_MAX_TOKENS = 20_000`, images and audio dropped
`codex-rs/core/src/compact_remote_v2_installation.rs`	remote compaction output; optional `CompactionImageBudget`

File	Relevant content
`codex-rs/rollout/src/compression.rs`	zstd level 3; 7-day idle threshold; `materialize_rollout_for_append`
`codex-rs/core/src/client.rs`	incremental requests (`previous_response_id`) and the conditions for a full send

Supporting reference: zstd `lib/compress/clevels.h`, level 3 parameters (`windowLog` 21, a 2 MiB window).